# Lecture 28 — Strings I: Polynomial Hashing, Prefix Function (KMP), Z-Function

> Year Curriculum · Prerequisites: Week 2 (C++ strings), Week 7 (prefix sums), Week 11 (modular arithmetic), Week 5 (binary search) · Estimated time: 2.5 hours

## 1. Overview

String problems in contests mostly reduce to a handful of primitive questions: *does pattern p occur in text t, and where?*, *are these two substrings equal?*, *what are the borders and periods of s?* Naively, comparing two strings of length k costs O(k), and searching for p in t costs O(|t|·|p|) in the worst case — 10^12 operations for strings of length 10^6. This lecture introduces three linear-time tools that every competitor must know:

1. **Polynomial hashing** turns any substring into a number in O(1) after O(n) preprocessing, so substring equality becomes an O(1) (probabilistic) check.
2. The **prefix function** (the heart of the Knuth–Morris–Pratt algorithm) records, for every prefix, its longest border. It gives O(n + m) pattern matching and direct access to borders and periods.
3. The **Z-function** records, for every position, the longest common prefix of the string with its suffix starting there. It solves the same family of problems with a different, often more convenient, viewpoint.

Next week (Strings II) builds tries, Aho–Corasick (prefix function for many patterns) and suffix arrays on top of these ideas.

## 2. Polynomial hashing

### 2.1 Definition

Fix a modulus M and a base B. The hash of `s = s[0] s[1] … s[k−1]` is

```
H(s) = s[0]·B^(k−1) + s[1]·B^(k−2) + … + s[k−1]  (mod M)
```

where characters are mapped to small positive integers (e.g., their ASCII codes). Equal strings always have equal hashes. If B is chosen uniformly at random, two **different** strings of length k collide with probability at most about k/M, because their difference is a non-zero polynomial of degree < k in B, which has at most k−1 roots modulo a prime M.

## 2.2 Prefix hashes and substring hashes

Like prefix sums, precompute

```
h[0] = 0,   h[i+1] = h[i]·B + s[i]   (mod M)
pw[0] = 1,  pw[i+1] = pw[i]·B        (mod M)
```

Then the hash of the half-open substring `s[l..r)` is

```
hash(l, r) = h[r] − h[l]·B^(r−l)   (mod M)
```

because `h[r]` contains `h[l]` "shifted left" by `r − l` positions. For `s = abcd`, `h[3] = a·B² + b·B + c` and `h[1] = a`, so `h[3] − h[1]·B² = b·B + c`, the hash of `bc`. Hashes of substrings of two different strings are comparable if both use the same B and M.

## 2.3 Choosing B and M safely

- **Do not use mod 2^64** (natural unsigned overflow). The Thue–Morse construction produces two different strings of length about 2000 with equal hashes for every odd base, and such tests are standard on Codeforces.
- **A single 30-bit prime with a fixed base** fails in two ways: an adversary can construct collisions for a known base, and, even with random data, comparing many substrings hits the birthday bound — with 10^5 hashes among ~10^9 values, a collision is likely.
- **Good choices:** two independent 30-bit primes (e.g., 10^9+7 and 10^9+9) with random bases, or the Mersenne prime **M = 2^61 − 1** with a random base. With 2^61 − 1 the collision probability per comparison is about n/2^61, negligible even for 10^7 comparisons. Multiplication uses `__uint128_t`, and the reduction exploits 2^61 ≡ 1 (mod M): split the 122-bit product into the low 61 bits and the rest, and add them.
- **Choose the base at runtime** (e.g., `mt19937_64` seeded by the clock). A fixed base can be attacked by hackers who read your code.

## 2.4 Applications

- **Substring equality** in O(1): the main use; it turns many O(n²)-comparison brute forces into O(n²) hash comparisons or better.
- **Periods (CSES 1733 *Finding Periods*).** `p` is a period of s iff `s[i] = s[i+p]` for all valid i, i.e., `s[0..n−p) == s[p..n)`. One hash comparison per p: O(n) total.
- **Longest common prefix** of two suffixes: binary search on the length with hash comparisons, O(log n). This also allows comparing two substrings lexicographically in O(log n) (compare the character after the LCP).
- **Palindromes**: hash s and reverse(s); `s[l..r)` is a palindrome iff its hash equals the hash of the corresponding reversed range. With binary search this finds the longest palindrome

centred at each position (CSES 1111, though Manacher's algorithm is the deterministic O(n) solution — Regionals track).
- **Distinct substrings** of a fixed length: insert hashes into a set or sort them (Codeforces 271D *Good Substrings*).

The drawback: hashing is **probabilistic**. A collision silently gives a wrong answer. The deterministic algorithms below are preferable when they apply.

# 3. Borders and the prefix function

## 3.1 Definitions

A **border** of a string is a proper prefix that is also a suffix. `abcababcab` has borders `ab` and `abcab`. The **prefix function** of s is the array

```
π[i] = length of the longest border of s[0..i]
```

For `s = abcababcab`: `π = 0 0 0 1 2 1 2 3 4 5`.

## 3.2 The border chain

**Lemma.** If `b` is the longest border of a string `u`, then the borders of `u` are exactly `b` and the borders of `b`. (A shorter border of `u` is a prefix and a suffix of `u`, hence also a prefix and a suffix of `b`.) Therefore all border lengths of `s[0..i]` are

```
π[i], π[π[i] − 1], π[π[π[i] − 1] − 1], …   until 0.
```

## 3.3 Computing π in O(n)

Suppose we know `π[0..i−1]` and want `π[i]`. A border of `s[0..i]` of length `k + 1` consists of a border of `s[0..i−1]` of length `k` followed by `s[i] = s[k]`. So try the border lengths of `s[0..i−1]` from longest to shortest — `k = π[i−1]`, then `π[k−1]`, … — and take the first `k` with `s[k] = s[i]`, giving `π[i] = k + 1`; if none works and `s[0] ≠ s[i]`, `π[i] = 0`.

**Amortized analysis.** `π[i] ≤ π[i−1] + 1`, so the value increases by at most 1 per step, while each iteration of the inner `while` decreases it by at least 1. The total number of decreases is at most the total number of increases, at most n. Hence O(n) overall.

**Trace** on `abcababcab` at i = 5 (`a`): `k = π[4] = 2`, `s[2] = c ≠ a`, fall back to `k = π[1] = 0`, `s[0] = a` matches, so `π[5] = 1`.

### 3.4 KMP pattern matching

To find pattern p (length m) in text t, either compute the prefix function of `p + '#' + t` (with `#` a character occurring in neither) and report positions where `π = m`, or — more memory-efficient — keep the state `k` = length of the longest prefix of p that is a suffix of the text read so far, and update it with the same fallback loop as above using `π` of p. When `k` reaches m, an occurrence ends at the current position; continue with `k = π[m−1]` so that overlapping occurrences are found. Total O(|t| + |p|) time and O(|p|) memory. This state machine is also the "KMP automaton" used in DP over strings, e.g., counting strings of length n that avoid a pattern, or Codeforces 808G *Anthem of Berland*, which maximizes the number of pattern occurrences with a DP over (position, automaton state).

### 3.5 Borders and periods from π

- All borders of s: the chain `π[n−1], π[π[n−1]−1], …` (CSES 1732 *Finding Borders*).
- `p` is a period of s iff `n − p` is a border length (or p = n). So the **smallest period** is `n − π[n−1]`, and all periods are `n − b` over the border chain, plus n.
- s is a repetition `u^k` with k ≥ 2 iff `π[n−1] > 0` and `n` is divisible by `n − π[n−1]` (Codeforces 1200E and many compression problems).
- **Codeforces 126B *Password***: find the longest substring that is a prefix, a suffix, and also occurs strictly inside. Candidate lengths are borders; a border of length b occurs inside iff some `π[i] ≥ b` for `i < n−1`.

## 4. The Z-function

### 4.1 Definition

```
z[i] = length of the longest common prefix of s and s[i..]     (i ≥ 1),  z[0] = 0 by convention
```

For `s = aabxaab`: `z = 0 1 0 0 3 1 0`.

### 4.2 Computing Z in O(n)

Maintain the segment `[l, r)` with the largest `r` found so far such that `s[l..r)` matches the prefix `s[0..r−l)`. For a new position `i`:

- If `i < r`, then `s[i..r)` equals `s[i−l..r−l)`, so `z[i] ≥ min(r − i, z[i − l])`. Start from that value.
- Extend `z[i]` by direct character comparisons.
- If `i + z[i] > r`, set `[l, r) = [i, i + z[i])`.

Each successful comparison in the extension step increases `r` (or happens at most once per `i` before failing), so the total work is O(n).

### 4.3 Applications

- **Matching:** compute Z of `p + '#' + t`; an occurrence starts at text position `i − m − 1` whenever `z[i] = m`.
- **Borders:** `k` is a border length iff `z[n−k] = k` (CSES 1732).
- **Periods:** `p < n` is a period iff `z[p] = n − p` (CSES 1733).
- **Prefix occurrence counts** (Codeforces 432D *Prefixes and Suffixes*): how many times each prefix occurs — count Z values and take suffix sums.
- **String compression:** the smallest `p` dividing n with `z[p] = n − p`.

### 4.4 Prefix function or Z-function?

Both run in O(n), and each can be converted into the other in O(n). The prefix function processes a stream character by character (useful for automata and for matching against a text that is not stored), while the Z-function often states border/period conditions more directly. Use whichever makes the condition simpler; many competitors know both by heart.

## 5. Complexity summary

| Tool | Preprocessing | Query | Deterministic? |
|---|---|---|---|
| Naive matching | — | O(nm) | yes |
| Polynomial hashing | O(n) | O(1) substring compare | no (tiny error probability) |
| Prefix function / KMP | O(m) | O(n) scan | yes |
| Z-function | O(n + m) | — | yes |

## 6. Reference implementations

All programs compile with `g++ -std=c++17 -O2 -Wall`, reproduce the CSES samples, and were cross-checked against brute force on 300 random strings over `{a, b}`.

### 6.1 Prefix function and KMP (CSES 1753 *String Matching*)

```cpp
#include <bits/stdc++.h>
using namespace std;

// CSES 1753 - String Matching: count occurrences of pattern p in text t.
// Prefix function: pi[i] = length of the longest proper border of s[0..i].
vector<int> prefixFunction(const string& s) {
```

```
    int n = s.size();
    vector<int> pi(n, 0);
    for (int i = 1; i < n; i++) {
        int k = pi[i - 1];
        while (k > 0 && s[i] != s[k]) k = pi[k - 1];   // fall back along borders
        if (s[i] == s[k]) k++;
        pi[i] = k;
    }
    return pi;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string t, p;
    cin >> t >> p;
    vector<int> pi = prefixFunction(p);
    int m = p.size(), k = 0, count = 0;
    for (char c : t) {                        // KMP automaton over the text
        while (k > 0 && c != p[k]) k = pi[k - 1];
        if (c == p[k]) k++;
        if (k == m) {
            count++;                          // match ends here
            k = pi[k - 1];                    // continue (overlaps allowed)
        }
    }
    cout << count << '\n';
}
```

## 6.2 Z-function (CSES 1732 *Finding Borders*)

```
#include <bits/stdc++.h>
using namespace std;

// CSES 1732 - Finding Borders, using the Z-function.
// z[i] = length of the longest common prefix of s and s[i..]; z[0] = 0 by convention.
vector<int> zFunction(const string& s) {
    int n = s.size();
    vector<int> z(n, 0);
    for (int i = 1, l = 0, r = 0; i < n; i++) {   // [l, r) = rightmost match window
        if (i < r) z[i] = min(r - i, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
        if (i + z[i] > r) { l = i; r = i + z[i]; }
    }
    return z;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    cin >> s;
    int n = s.size();
```

```
    vector<int> z = zFunction(s);
    // k is a border length iff the suffix of length k equals the prefix: z[n-k] == k.
    for (int k = 1; k < n; k++)
        if (z[n - k] == k) cout << k << ' ';
    cout << '\n';
}
```

## 6.3 Hashing mod 2^61 − 1 (CSES 1733 *Finding Periods*)

```
#include <bits/stdc++.h>
using namespace std;

// CSES 1733 - Finding Periods, using polynomial hashing mod 2^61 - 1.
// p is a period of s iff s[0..n-p) == s[p..n), i.e. one O(1) hash comparison.
typedef unsigned long long ull;
const ull MOD = (1ULL << 61) - 1;

ull mulmod(ull a, ull b) {
    __uint128_t c = (__uint128_t)a * b;
    ull lo = (ull)(c & MOD), hi = (ull)(c >> 61);
    ull r = lo + hi;
    if (r >= MOD) r -= MOD;
    return r;
}

struct Hasher {
    vector<ull> h, pw;                      // h[i] = hash of s[0..i)
    Hasher(const string& s, ull base) : h(s.size() + 1, 0), pw(s.size() + 1, 1) {
        for (size_t i = 0; i < s.size(); i++) {
            h[i + 1] = mulmod(h[i], base) + (ull)(unsigned char)s[i];
            if (h[i + 1] >= MOD) h[i + 1] -= MOD;
            pw[i + 1] = mulmod(pw[i], base);
        }
    }
    ull get(int l, int r) const {           // hash of s[l..r)
        ull res = h[r] + MOD - mulmod(h[l], pw[r - l]);
        if (res >= MOD) res -= MOD;
        return res;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    cin >> s;
    int n = s.size();
    mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
    ull base = rng() % (MOD - 1000) + 500;   // random base defeats anti-hash tests
    Hasher H(s, base);
    for (int p = 1; p <= n; p++)
        if (H.get(0, n - p) == H.get(p, n)) cout << p << ' ';
```

```
    cout << '\n';
}
```

## 7. Common mistakes

- **Weak hashing.** mod 2^64, a fixed base, or a single small modulus — all are hackable or collide by the birthday bound.
- **Unsigned underflow.** In `h[r] − h[l]·pw[r−l]`, add M before subtracting (as in `get` above).
- **Signed characters.** `char` may be negative for non-ASCII input; cast through `unsigned char`. Also avoid mapping any character to 0 (then `a` and `aa` both hash to 0 when compared across different lengths).
- **Prefix function fallback.** It is `k = π[k−1]`, not `π[k]`.
- **Missing overlapping matches** in KMP by resetting `k = 0` after a match instead of `k = π[m−1]`.
- **Separator collisions.** The `#` in `p + '#' + t` must not occur in either string; otherwise matches can span the separator.
- **Z-function `z[0]`.** By convention 0 (some write n). Be consistent, especially when testing borders with `z[n−k]`.
- **Quadratic Z implementation**: forgetting the `min(r − i, z[i − l])` initialization leaves correct but O(n²) code that times out on `aaaa…a`.

## 8. Practice problems

Warm-up:

- CSES 1753 — *String Matching* — https://cses.fi/problemset/task/1753
- CSES 1732 — *Finding Borders* — https://cses.fi/problemset/task/1732
- CSES 1733 — *Finding Periods* — https://cses.fi/problemset/task/1733

Core:

- Codeforces 126B — *Password* — https://codeforces.com/problemset/problem/126/B
- Codeforces 432D — *Prefixes and Suffixes* — https://codeforces.com/problemset/problem/432/D
- Codeforces 471D — *MUH and Cube Walls* — https://codeforces.com/problemset/problem/471/D
- Codeforces 271D — *Good Substrings* — https://codeforces.com/problemset/problem/271/D

Challenge:

- Codeforces 1200E — *Compress Words* — https://codeforces.com/problemset/problem/1200/E
- CSES 1110 — *Minimal Rotation* — https://cses.fi/problemset/task/1110

- CSES 1111 — *Longest Palindrome* — https://cses.fi/problemset/task/1111
- Codeforces 808G — *Anthem of Berland* (KMP automaton + DP) — https://codeforces.com/problemset/problem/808/G

## 9. Further reading

- CP-Algorithms: "String Hashing" (https://cp-algorithms.com/string/string-hashing.html), "Prefix function — Knuth–Morris–Pratt" (https://cp-algorithms.com/string/prefix-function.html), "Z-function" (https://cp-algorithms.com/string/z-function.html).
- "Anti-hash test" and Thue–Morse discussion on Codeforces — https://codeforces.com/blog/entry/4898
- Antti Laaksonen, *Competitive Programmer's Handbook*, Chapter 26 "String algorithms".
- D. Knuth, J. Morris, V. Pratt, "Fast Pattern Matching in Strings", *SIAM J. Computing*, 1977.
- Next week: Strings II — tries, Aho–Corasick (the prefix function generalized to many patterns), and suffix arrays.